ARQUITETURA DO SOFTWARE

Visão geral da arquitetura

Será utilizada a arquitetura de 3 camadas (ou *three-tier architecture*), dada a simplicidade e a familiaridade com essa estrutura, a fim de facilitar o desenvolvimento e torná-lo mais modular e menos acoplado. Essas três camadas são:

Camada de apresentação (*Presentation layer*)

Essa será a camada responsável pela interface de usuário, onde ele irá interagir com as telas e seus respectivos componentes e funcionalidades. A tecnologia escolhida para essa camada foi o *React Native*, dado a sua flexibilidade em ser multiplataforma.

A princípio, utilizaremos os componentes padrão do *React Native,* a fim de garantir que, o mesmo componente seja compatível com outros sistemas operacionais. Bibliotecas de terceiros serão usadas, se necessário, e caso seja, será feito um levantamento a respeito da compatibilidade do(s) componente(s), dada a possibilidade de problemas referentes a isso, uma vez que será utilizada uma tecnologia multiplataforma.

Os principais componentes que serão utilizados no desenvolvimento dessa camada, são os *TextInputs*, *Switches*, *Buttons*, *Text*, *ScrollViews,* entre outros componentes vistos em aula e encontrados ao longo de pesquisas e leitura de documentações. Adicionalmente, foram obtidas as fichas que são utilizadas para o pré-diagnóstico de crianças com indícios de ansiedade, o que indica ainda mais a necessidade de uma interface gráfica lúdica, simples, intuitiva e confortável de utilizar, especialmente ao considerar que os usuários precisarão preencher formulários extensos. Para isso, pretende-se adotar uma abordagem baseada em etapas, dividindo o preenchimento em múltiplas telas, evitando uma tela "densa", com excesso de informações e componentes em um único lugar, o que poderia dificultar a navegação e tornar a experiência de uso cansativa.

Camada de aplicação/lógica de negócios (***Business layer***)

A camada da aplicação/lógica de negócios contempla as regras do sistema e a parte geral de processamento de dados. Nessa camada, estarão presentes as funções responsáveis pela autenticação, autorização, validações e comunicação com o banco de dados. O desenvolvimento será voltado a uma *API* (*Application Programming Interface*) RESTful utilizando *Node.js* com o *framework NestJS*, dado a sua estrutura organizada e modular.

As regras já estabelecidas com modelagens e diagramações anteriores, têm como base, dois exemplares de fichas usadas no pré-diagnóstico de crianças com indícios de ansiedade, somadas a regras mais gerais, como impedir que um formulário tenha perguntas editadas/incluídas/excluídas após já ter sido respondido, registrar ações de usuários para fins de auditoria, entre outros.

Quanto à hospedagem desta *API*, o Render foi a plataforma escolhida, por já ter sido utilizado em aula, viabilizando maior familiaridade com a plataforma e com o seu funcionamento.

Camada de dados (***Data layer***)

Para esta camada, utilizaremos um banco de dados relacional por meio do *PostgreSQL*, hospedado no Neon, também já utilizado em aula e em outros projetos acadêmicos. A comunicação da *API* com o banco, será feita com o *ORM (Object-Relational Mapping,* ou Mapeamento Objeto-Relacional*) Prisma*.

Nota: dado a fase inicial do projeto, o local onde o banco será hospedado pode mudar, de acordo com a necessidade e as dimensões que o projeto for tomando ao longo de seu desenvolvimento. Por ora, o que teve maior peso na decisão para hospedagem do banco, foi a plataforma que possui menos limitações do plano gratuito.

Tecnologias e ferramentas utilizadas

Para viabilizar a implementação do modelo arquitetural neste projeto, foram selecionadas tecnologias voltadas ao desenvolvimento mobile, à construção de *APIs RESTful*, ao acesso a dados e à infraestrutura da aplicação que fossem gratuitas e que sem tantas limitações restritivas para o escopo aqui elaborado. Dessa forma, serão utilizados o *React Native* no desenvolvimento da

interface gráfica, o *Node.js* com o *framework NestJS* na camada de aplicação e lógica de negócios, o *PostgreSQL* como sistema gerenciador de banco de dados relacional e o *Prisma* como *ORM* responsável pela comunicação entre a *API* e o banco de dados. Para hospedagem da *API* e do banco de dados, serão consideradas as plataformas Render e Neon, respectivamente.

Ressalta-se que outras tecnologias e ferramentas poderão ser adotadas ao longo do desenvolvimento do projeto, conforme novas necessidades sejam identificadas. As tecnologias apresentadas refletem o conhecimento e os requisitos atuais da equipe, podendo ser complementadas ou substituídas à medida que o projeto evoluir.

JUSTIFICATIVA PARA A ESCOLHA DA TECNOLOGIA

Camada de apresentação (*Presentation layer*)

O *React Native* utiliza o *JavaScript* como uma de suas bases, o que facilita o desenvolvimento e a compreensão do código-fonte, além de um maior apoio, uma vez que, essa linguagem, por ser amplamente utilizada no mercado, possui uma comunidade grande e consolidada de desenvolvedores, quando comparada a outras, como o *Flutter*. proporciona uma familiaridade maior, tendo em vista os conteúdos repassados em aula e a experiência profissional da equipe que desenvolverá o projeto.

A escolha pela utilização dos componentes nativos se dá pela garantia de uma maior compatibilidade entre as diferentes plataformas, uma vez que, ao trabalhar com tecnologias do tipo, é de suma importância analisar a conformidade entre as plataformas envolvidas.

Camada de aplicação/lógica de negócios (*Business layer*)

A escolha do *Node.js* e do *NestJS* teve como base não apenas os conteúdos abordados em aula, mas também, a facilidade de uso e a familiaridade da equipe com tecnologias base e/ou semelhantes utilizadas nessa *stack*, contribuindo para uma maior produtividade ao longo do projeto.

Além disso, o *Node.js* é amplamente utilizado no desenvolvimento de *APIs RESTful* e possui uma comunidade bastante ativa, oferecendo uma grande quantidade de documentação, exemplos e materiais de apoio. Já o *NestJS* foi escolhido por sua estrutura organizada e modular, além de recursos nativos que facilitam a manutenção, escalabilidade e evolução da aplicação ao longo do tempo.

Outro fator considerado foi a similaridade do *NestJS* com tecnologias já conhecidas pela equipe, especialmente o ecossistema *.NET*. Conceitos como programação orientada a objetos, injeção de dependência, utilização de *controllers*, *services* e *decorators* (*attributes* do *.NET*) tornam a curva de aprendizado mais suave para desenvolvedores habituados a esse modelo arquitetural, favorecendo a adoção de boas práticas de desenvolvimento, desde o início do projeto.

Camada de dados (*Data layer*)

Para a camada de dados, optou-se pela utilização do *PostgreSQL*, dado a sua ampla utilização no mercado, o que confere uma ampla comunidade de desenvolvedores, resultando em um maior suporte conforme o projeto avança, e novas necessidades surgem. A escolha por um banco de dados relacional está alinhada às necessidades do projeto, que envolve relacionamentos entre entidades como usuários, formulários, perguntas, respostas e registros de auditoria, e a ideia de uso futuro, de ferramentas para análises preditivas, conforme o volume de dados aumenta.

Para a comunicação entre a aplicação e o banco de dados, será utilizado o *Prisma ORM*, que simplifica o acesso aos dados e contribui para uma maior produtividade, dado que a utilização de um *ORM* facilita o mapeamento de entidades, mitigando problemas de tipagem e validações, reduzindo assim, erros durante a implementação.

Quanto à hospedagem, será considerada a plataforma *Neon*, já utilizada em atividades acadêmicas, totalmente compatível com *PostgreSQL* e com menores limitações no plano gratuito, o que acaba sendo um fator de peso na decisão de qual serviço será utilizado, dada a fase inicial do projeto.